TP3 - Certificats SSL/TLS

Mathieu WAHARTE - APP5

On va créer une autorité de certification locale afin de pouvoir signer ensuite le certificat du serveur Apache. Le but est de reproduire en local le fonctionnement classique d'une chaîne de confiance TLS.

1 - Génération de certificats pour apache

Le répertoire `ssl` contient les fichiers de clé privée, de certificat et de demande de signature pour l'autorité de certification (CA) et pour le serveur Apache. Voici un aperçu de la structure du répertoire :

```
bluefir  TP3  main  cd /etc/ssl
bluefir  ssl  ls
certs  openssl.cnf  private
```

Créons la clé privée du CA avec `openssl genrsa -des3 -out private/ca.pem 2048` (et pass_phrase) :

```
bluefir  ssl  sudo openssl genrsa -des3 -out private/ca.pem 2048
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
```

On va ensuite modifier une copie du fichier de configuration `openssl.cnf` pour y ajouter les informations de notre CA :

```
[ req_distinguished_name ]
countryName                     = Country Name (2 letter code)
countryName_default             = FR
countryName_min                 = 2
countryName_max                 = 2

stateOrProvinceName             = State or Province Name (full name)
stateOrProvinceName_default     = Essonne

localityName                    = Locality Name (eg, city)
localityName_default            = Orsay

0.organizationName              = Organization Name (eg, company)
0.organizationName_default      = Polytech PS
```

Le nom de domaine de la machine peut être obtenu avec la commande `hostname -f` et est : `LAPTOP-QJHCANDR.localdomain`.

On peut ensuite créer le certificat à partir de ce fichier de configuration avec `openssl req -new -x509 -days 365 -key private/ca.pem -out private/ca.crt -config essai.cnf`:

```
bluefir  ssl  sudo openssl req -new -x509 -days 365 -key private/ca.pem -out private/ca.crt -config essai.cnf
Enter pass phrase for private/ca.pem:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [FR]:
State or Province Name (full name) [Essonne]:
Locality Name (eg, city) [Orsay]:
Organization Name (eg, company) [Polytech PS]:
Organizational Unit Name (eg, section) []:APP5_IIM
Common Name (e.g. server FQDN or YOUR name) []:LAPTOP-QJHCANDR.localdomain
Email Address []:.
```

Le CommonName renseigné est `LAPTOP-QJHCANDR.localdomain`.

1. Pour afficher le certificat de l'autorité de certification, on utilise la commande `openssl x509 -noout -text -in private/ca.crt`. On obtient alors le détail complet du certificat X.509 du CA, avec notamment le sujet, l'émetteur, la période de validité, la clé publique et les extensions du certificat :

```
bluefir  ssl  sudo openssl x509 -noout -text -in private/ca.crt
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            23:62:7f:74:fd:5a:75:1f:28:c2:ab:df:da:3a:d1:80:68:35:8d:44
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = FR, ST = Essonne, L = Orsay, O = Polytech PS, OU = APP5_IIM, CN = LAPTOP-QJHCANDR.localdomain
        Validity
            Not Before: Mar 12 15:09:52 2026 GMT
            Not After : Mar 12 15:09:52 2027 GMT
        Subject: C = FR, ST = Essonne, L = Orsay, O = Polytech PS, OU = APP5_IIM, CN = LAPTOP-QJHCANDR.localdomain
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:bb:63:50:22:dc:ad:91:94:9a:69:05:4f:2a:8e:
                    6d:95:44:82:d3:9e:bf:36:bf:98:16:aa:68:1d:fe:
                    1c:b6:26:e1:47:6e:16:18:b6:f7:d0:9a:5d:2c:14:
                    0e:f4:8b:de:a9:5d:84:7a:9e:6a:ba:a5:3d:f5:30:
                    46:a2:7d:0f:86:57:6c:33:47:2f:27:39:87:b3:00:
                    6c:fe:b8:ef:bb:d9:0d:68:b9:63:cd:96:43:c0:b0:
                    5d:11:33:b2:ca:ab:a4:14:98:86:39:4d:1e:30:66:
                    c1:4b:39:d6:3c:9a:80:6e:ce:7a:01:74:e1:b3:1c:
```

Le common name avec le nom de domaine dns complet (obtensible avec `hostname -f`) est : `LAPTOP-QJHCANDR.localdomain` ce qui matche le Common Name renseigné.

`openssl req -nodes -new -keyout private/server.key -out private/server.csr -config essai.cnf` permet de générer la clé privée du serveur et la demande de signature (CSR) en une

seule commande :

```
bluefir  ssl  sudo openssl req -nodes -new -keyout private/server.key -out private/server.csr -config essai.cnf
.....+.+..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*..+..+....+..+.....+.+..+....+......+.....++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++*..+.......+..+......+...+....+.............+..+.+..+.+..+.....+...+.......+.....+.+.....+...+...+..+.....
........+...+.....+.........................+.....+....+......+...........+...+.+..+.........+..+.+..+....++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++
.....+.............+..+.........+...+.....+.+.....+....+...+..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*....+......+++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++*.+........+...+......+........+........+...+.....+...+...+.......+.....+...+...+....+...+.............
.............+..+.........+.+......+...+............+..+.+..............+.............+............+........+......+.........+...+..+................+..
...+............+.+.........+......+.....+.+.........+.....+......+.........+......+.........+.+...+........+...+.....................+...+++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [FR]:
State or Province Name (full name) [Essonne]:
Locality Name (eg, city) [Orsay]:
Organization Name (eg, company) [Polytech PS]:
Organizational Unit Name (eg, section) []:APP5_IIM
Common Name (e.g. server FQDN or YOUR name) []:LAPTOP-QJHCANDR.localdomain
Email Address []:.

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

2. Le common name que j'ai spécifié est `LAPTOP-QJHCANDR.localdomain`.
   Pour afficher la demande de certificat, on utilise ensuite `openssl req -noout -text -in private/server.csr`. Cette commande permet de vérifier les informations présentes dans la CSR, notamment le sujet et la taille de la clé publique. La sortie est à coller ici :

```
bluefir  ssl  sudo openssl req -noout -text -in private/server.csr
Certificate Request:
    Data:
        Version: 1 (0x0)
        Subject: C = FR, ST = Essonne, L = Orsay, O = Polytech PS, OU = APP5_IIM, CN = LAPTOP-QJHCANDR.localdomain
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:9d:62:07:0b:60:5d:f1:9d:fb:00:dc:21:f9:03:
                    8d:53:a8:7a:a4:41:3a:9f:b2:27:4f:91:ce:49:b5:
                    18:d7:b4:09:dd:c6:a8:37:9c:47:74:0a:c5:fc:a9:
                    60:d4:78:d9:0f:8d:cb:9c:1b:85:02:6a:a3:75:cd:
                    95:4f:7b:86:34:b2:f1:06:84:81:90:a6:5b:08:31:
                    20:e3:f9:fd:f6:0d:f5:8d:a2:cb:24:c7:da:9e:f0:
                    5e:39:0e:7b:c3:ad:75:c0:08:9a:a0:66:d3:02:de:
                    11:b8:fc:d2:02:e1:c1:6f:55:47:3c:ce:e7:54:cd:
                    aa:f9:a6:21:f5:b2:89:59:24:1c:e5:cc:b7:f3:67:
                    4d:67:42:a1:c8:c3:04:b5:0b:d7:e2:2a:5f:f7:f8:
                    c9:06:73:9e:02:53:8b:7c:6c:a0:8a:03:3c:e2:92:
                    d7:fb:26:e0:8a:a7:cd:1e:51:ef:8b:e0:fe:88:80:
                    70:b2:82:ca:2d:91:03:61:c6:87:c4:a2:90:14:b9:
                    27:0b:cc:b5:fd:f5:2b:79:23:88:64:41:b1:8f:49:
                    74:8e:46:5c:9e:d5:e0:7d:4f:32:83:2b:65:5e:9e:
                    0c:cf:f4:84:8f:16:75:d4:fa:e7:08:50:fe:c7:f2:
                    04:52:ff:6d:30:4b:6e:37:16:56:0c:f3:04:c8:95:
                    f3:c9
                Exponent: 65537 (0x10001)
        Attributes:
```

3. Dans la demande de certificat, on observe que la taille de la clé RSA utilisée est normalement de **2048 bits**. On retrouve cette information dans une ligne du type `Public-Key: (2048 bit)`.

4. Cette option peut être changée directement au moment de la génération de la clé. Par exemple, dans une commande comme `openssl genrsa -des3 -out private/ca.pem 2048`, la valeur `2048` correspond à la taille de la clé. On peut donc la remplacer par `4096` si l'on veut une clé plus grande. De manière générale, c'est au moment de la génération de la clé RSA que l'on choisit cette taille.

On crée le répertoire `demoCA`, le fichier `serial` avec la valeur `01`, le répertoire `newcerts` et le fichier `index.txt` vide.
Ensuite, on signe la demande de certificat du serveur avec la commande `openssl ca -out private/server.crt -in private/server.csr -cert private/ca.crt -keyfile private/ca.pem` :

```
bluefir  ssl  sudo openssl ca -out private/server.crt -in private/server.csr -cert private/ca.crt -keyfile private/ca.pem
Using configuration from /usr/lib/ssl/openssl.cnf
Enter pass phrase for private/ca.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 2 (0x2)
        Validity
            Not Before: Mar 12 15:13:14 2026 GMT
            Not After : Mar 12 15:13:14 2027 GMT
        Subject:
            countryName               = FR
            stateOrProvinceName       = Essonne
            organizationName          = Polytech PS
            organizationalUnitName    = APP5_IIM
            commonName                = LAPTOP-QJHCANDR.localdomain
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                07:D8:05:3A:60:CD:35:4A:CE:35:78:D3:D0:86:D1:FA:E2:55:ED:21
            X509v3 Authority Key Identifier:
                75:F6:CA:57:C0:47:D9:89:A0:95:20:78:78:44:2C:2C:11:E8:A2:36
Certificate is to be certified until Mar 12 15:13:14 2027 GMT (365 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
```

5. Le mot de passe demandé lors de la signature du certificat serveur est celui de la clé privée du CA, c'est-à-dire `ca.pem`. Il est demandé parce que cette clé a été créée avec l'option `-des3`, donc elle est chiffrée et protégée par mot de passe. OpenSSL a besoin de déverrouiller cette clé privée pour pouvoir signer le certificat du serveur.

6. Pour afficher le certificat du serveur, on utilise la commande `openssl x509 -noout -text -in private/server.crt`. On obtient alors le détail complet du certificat signé :

```
bluefir  ssl  sudo openssl x509 -noout -text -in private/server.crt
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 2 (0x2)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = FR, ST = Essonne, L = Orsay, O = Polytech PS, OU = APP5_IIM, CN = LAPTOP-QJHCANDR.localdomain
        Validity
            Not Before: Mar 12 15:13:14 2026 GMT
            Not After : Mar 12 15:13:14 2027 GMT
        Subject: C = FR, ST = Essonne, O = Polytech PS, OU = APP5_IIM, CN = LAPTOP-QJHCANDR.localdomain
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:9d:62:07:0b:60:5d:f1:9d:fb:00:dc:21:f9:03:
                    8d:53:a8:7a:a4:41:3a:9f:b2:27:4f:91:ce:49:b5:
                    18:d7:b4:09:dd:c6:a8:37:9c:47:74:0a:c5:fc:a9:
                    60:d4:78:d9:0f:8d:cb:9c:1b:85:02:6a:a3:75:cd:
                    95:4f:7b:86:34:b2:f1:06:84:81:90:a6:5b:08:31:
                    20:e3:f9:fd:f6:0d:f5:8d:a2:cb:24:c7:da:9e:f0:
                    5e:39:0e:7b:c3:ad:75:c0:08:9a:a0:66:d3:02:de:
                    11:b8:fc:d2:02:e1:c1:6f:55:47:3c:ce:e7:54:cd:
                    aa:f9:a6:21:f5:b2:89:59:24:1c:e5:cc:b7:f3:67:
                    4d:67:42:a1:c8:c3:04:b5:0b:d7:e2:2a:5f:f7:f8:
                    c9:06:73:9e:02:53:8b:7c:6c:a0:8a:03:3c:e2:92:
                    d7:fb:26:e0:8a:a7:cd:1e:51:ef:8b:e0:fe:88:80:
                    70:b2:82:ca:2d:91:03:61:c6:87:c4:a2:90:14:b9:
                    27:0b:cc:b5:fd:f5:2b:79:23:88:64:41:b1:8f:49:
                    74:8e:46:5c:9e:d5:e0:7d:4f:32:83:2b:65:5e:9e:
                    0c:cf:f4:84:8f:16:75:d4:fa:e7:08:50:fe:c7:f2:
                    04:52:ff:6d:30:4b:6e:37:16:56:0c:f3:04:c8:95:
```

2 - Mise en place d'un serveur apache https

Une fois le certificat serveur généré et signé, on peut configurer Apache pour activer HTTPS sur le port 443. Activons le module ssl pour apache2 avec `a2enmod ssl` :

```
bluefir  ssl  sudo a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

Puis activons le site par défaut en SSL avec `a2ensite default-ssl.conf` :

```
bluefir  ssl  sudo a2ensite default-ssl.conf
Enabling site default-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
```

Ensuite, on redémarre le service Apache pour appliquer les changements : `systemctl restart apache2`.

7. Les deux lignes de configuration concernant les certificats dans `default-ssl.conf` sont les suivantes :

```
#   A self-signed (snakeoil) certificate can be created by installing
#   the ssl-cert package. See
#   /usr/share/doc/apache2/README.Debian.gz for more info.
#   If both key and certificate are stored in the same file, only the
#   SSLCertificateFile directive is needed.
SSLCertificateFile      /etc/ssl/private/server.crt
SSLCertificateKeyFile   /etc/ssl/private/server.key
```

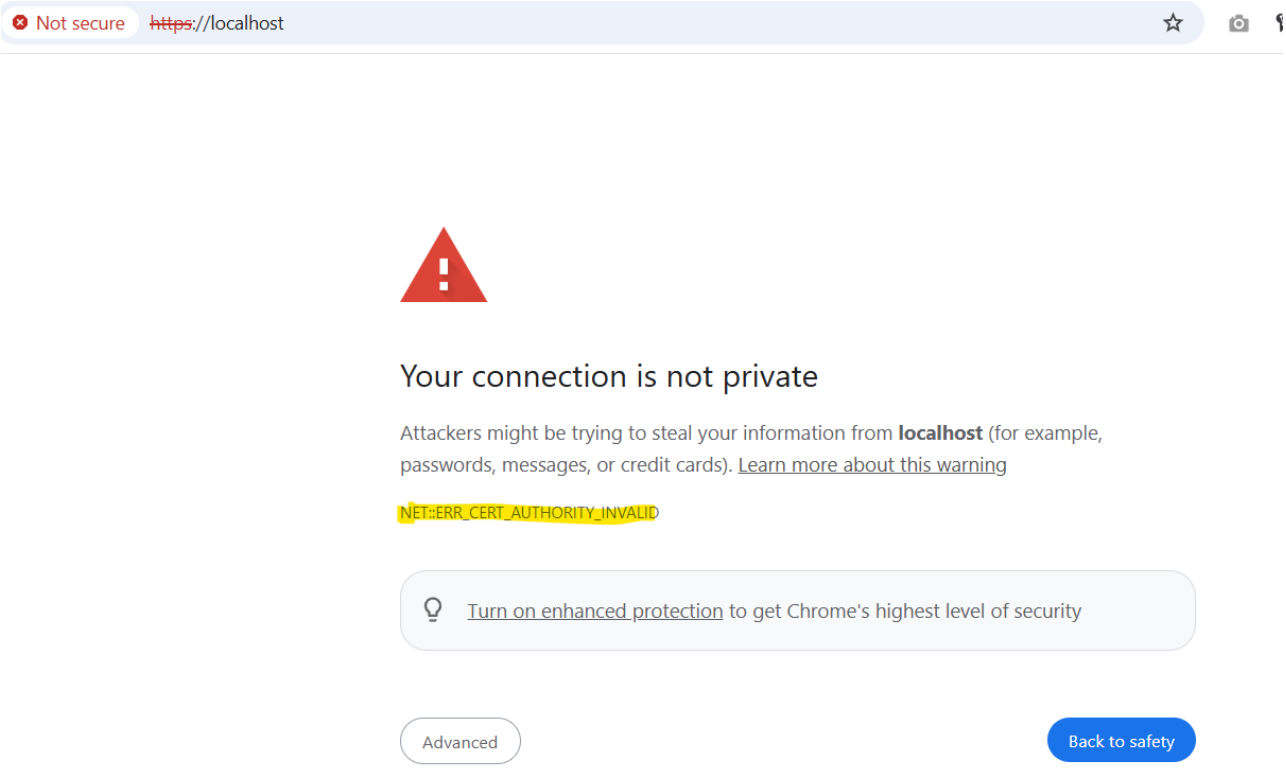
```
SSLCertificateFile /etc/ssl/private/server.crt
SSLCertificateKeyFile /etc/ssl/private/server.key
```

8. Pour vérifier que le service Apache2 en HTTPS est disponible sur la machine virtuelle, on peut par exemple utiliser la commande `curl -kI https://localhost`. Elle permet d'interroger le serveur

localement en ignorant l'erreur de certificat auto-signé:

```
bluefir  ssl  curl -kI https://localhost
HTTP/1.1 200 OK
Date: Thu, 12 Mar 2026 14:54:46 GMT
Server: Apache/2.4.66 (Debian)
Last-Modified: Thu, 12 Mar 2026 14:46:12 GMT
ETag: "29cd-64cd4d05397c7"
Accept-Ranges: bytes
Content-Length: 10701
Vary: Accept-Encoding
Content-Type: text/html
```

9. En accédant au site avec l'URL `https://localhost`, le navigateur remonte normalement deux erreurs. D'abord, le certificat n'est pas approuvé car il est signé par une autorité locale que le navigateur ne connaît pas. Ensuite, il y a généralement une erreur de nom, car le certificat a été généré pour le nom de domaine complet de la machine virtuelle et non pour `localhost`. C'est logique ici, puisque `localhost` ne correspond pas au nom déclaré dans le certificat.



Certificate Viewer: LAPTOP-QJHCANDR.localdomain

General Details

Issued To

Common Name (CN)	LAPTOP-QJHCANDR.localdomain
Organization (O)	<Not Part Of Certificate>
Organizational Unit (OU)	<Not Part Of Certificate>

Issued By

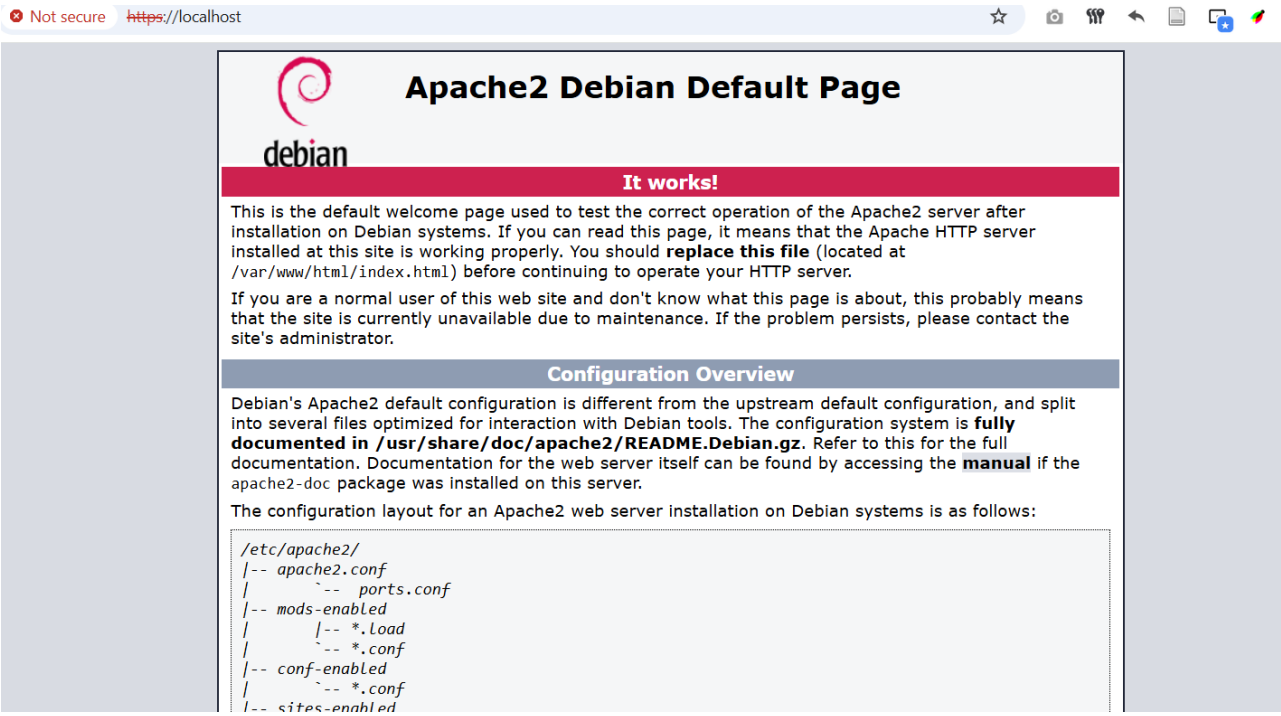
Common Name (CN)	LAPTOP-QJHCANDR.localdomain
Organization (O)	<Not Part Of Certificate>
Organizational Unit (OU)	<Not Part Of Certificate>

Validity Period

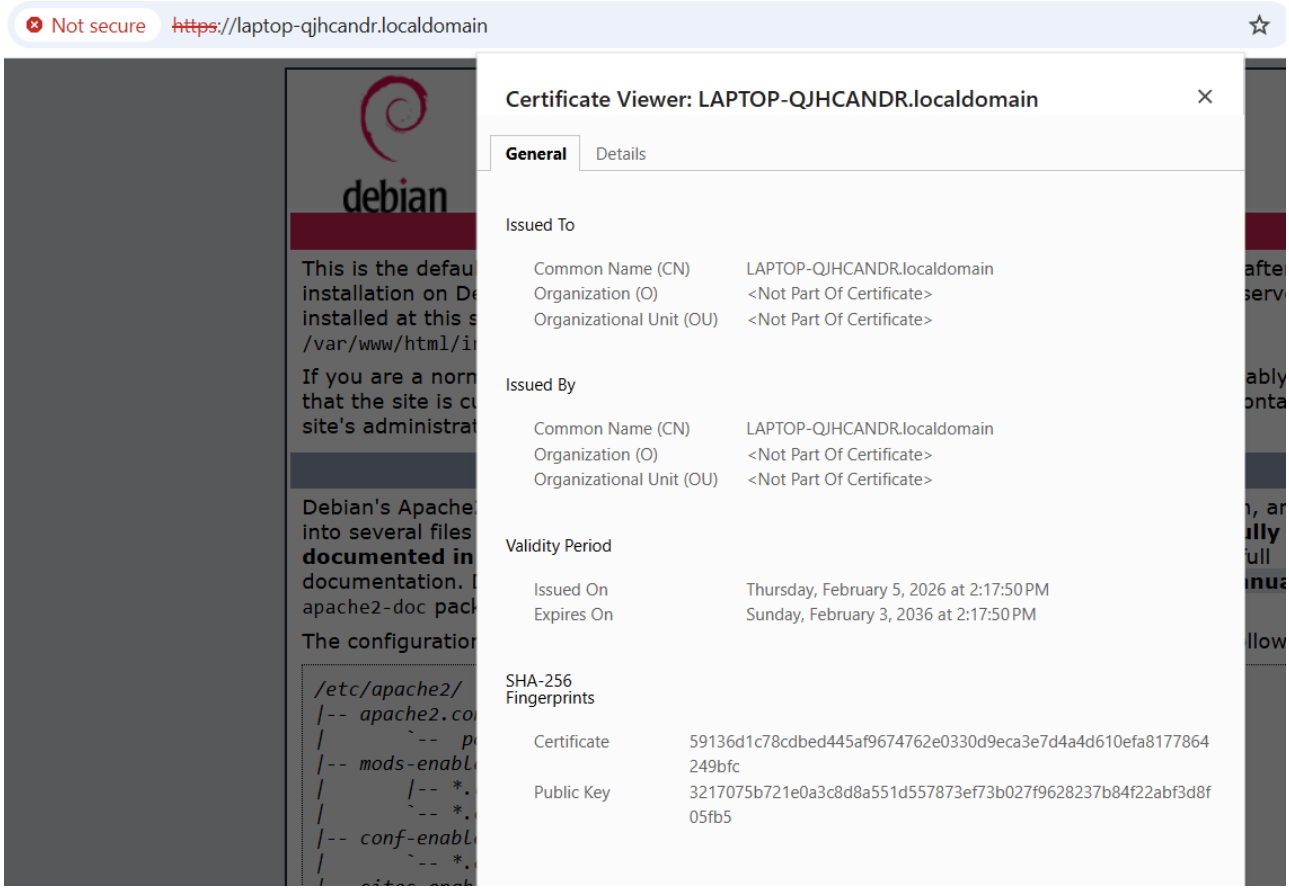
Issued On	Thursday, February 5, 2026 at 2:17:50 PM
Expires On	Sunday, February 3, 2036 at 2:17:50 PM

SHA-256 Fingerprints

Certificate	59136d1c78cdbed445af9674762e0330d9eca3e7d4a4d610efa8177864249bfc
Public Key	3217075b721e0a3c8d8a551d557873ef73b027f9628237b84f22abf3d8f05fb5



10. Pour l'accès avec le nom de domaine officiel, l'URL utilisée est : `https://LAPTOP-QJHCANDR.localdomain/`. Comme j'utilise WSL, lamachine n'est pas ajouté comme host dans mon navigateur, donc je dois ajouter une entrée dans le fichier `hosts` de Windows pour faire correspondre ce nom de domaine à l'adresse IP de la machine virtuelle. Par exemple, si l'adresse IP de la machine virtuelle est `172.27.x.x` (obtenue avec `hostname -I`).



Le navigateur affiche encore normalement un avertissement de sécurité car l'autorité de certification locale n'est pas reconnue par défaut. On corrige donc une partie du problème, mais pas encore la question de

confiance sur le CA.

11. Comme je n'ai pas accès aux machines virtuelles pour ce TP, je ne peux pas tester l'accès distant depuis les machines de la fac et donc je n'ai pas de configuration de proxy.

12. Pour supprimer les erreurs HTTPS, la solution consiste à faire reconnaître le certificat du CA local comme une autorité de confiance sur la machine cliente. Pour cela, il faut récupérer le fichier `ca.crt`, l'importer dans le navigateur ou dans le magasin de certificats du système ou navigateur comme autorité de confiance, puis accéder au site avec le bon nom DNS, c'est-à-dire exactement celui utilisé comme **Common Name** dans le certificat serveur. Une fois cela fait, le navigateur ne signale plus d'erreur de confiance, et si le nom de domaine est correct il n'y a plus non plus d'erreur de correspondance de nom. C'est donc bien l'import du certificat du CA qui permet de faire disparaître l'alerte principale. Sur Chrome, il suffit de drag and drop le fichier de certificat donc je n'ai pas de screenshot.

3 - Création et utilisation d'un certificat client

On reprend ensuite le même principe pour générer un certificat client, signé lui aussi par le CA local, on exécute donc `openssl req -nodes -new -keyout private/client.key -out private/client.csr -config essai.cnf` et `openssl ca -out private/client.crt -in private/client.csr -cert private/ca.crt -keyfile private/ca.pem` :

```
bluefir  ssl  sudo openssl req -nodes -new -keyout private/client.key -out private/client.csr -config essai.cnf
[sudo] password for bluefir:
..+.............+..........+...+.+..+...+....+......+.....+...+...+.......+.....+.+...+.....+.....+.+..+.....+....+..........+........+...+.......
..........+.............+...+..+.+..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*......+.....+++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++*..........+.....+....+.....+......+.+.....+.+.............+..+.+.............+.......+.....+..................+........+.
....+...+....+......+.......+....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
...+..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+......+.....+.+......+........+....+.....+.........+..+.+...+........+...+..+..
.+....+.....+......+...........+.+.............+............+....+.................+.+...+..+...........+..+.....+............+..+++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++*....+.........+.................+...+........+........+...+.....+..........+...+..............+..
..+..+.+..............+........+...+...+........+......+...........+.......................+.............+......+..+.+.....+.+.....+...+.......+..
...+...+...........+........+....+........+.....+.......+........+....+...+...........+...........+....+.....+....+...+++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [FR]:
State or Province Name (full name) [Essonne]:
Locality Name (eg, city) [Orsay]:
Organization Name (eg, company) [Polytech PS]:
Organizational Unit Name (eg, section) []:APP5_IIM
Common Name (e.g. server FQDN or YOUR name) []:John Doe
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

```
bluefir  ssl  sudo openssl ca -out private/client.crt -in private/client.csr -cert private/ca.crt -keyfile private/ca.pem
Using configuration from /usr/lib/ssl/openssl.cnf
Enter pass phrase for private/ca.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 3 (0x3)
        Validity
            Not Before: Mar 12 15:36:22 2026 GMT
            Not After : Mar 12 15:36:22 2027 GMT
        Subject:
            countryName               = FR
            stateOrProvinceName       = Essonne
            organizationName          = Polytech PS
            organizationalUnitName    = APP5_IIM
            commonName                = John Doe
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                14:46:EC:4D:EA:20:F0:AB:5E:D5:45:DA:6C:86:1E:0D:8C:46:8F:18
            X509v3 Authority Key Identifier:
                75:F6:CA:57:C0:47:D9:89:A0:95:20:78:78:44:2C:2C:11:E8:A2:36
Certificate is to be certified until Mar 12 15:36:22 2027 GMT (365 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
```

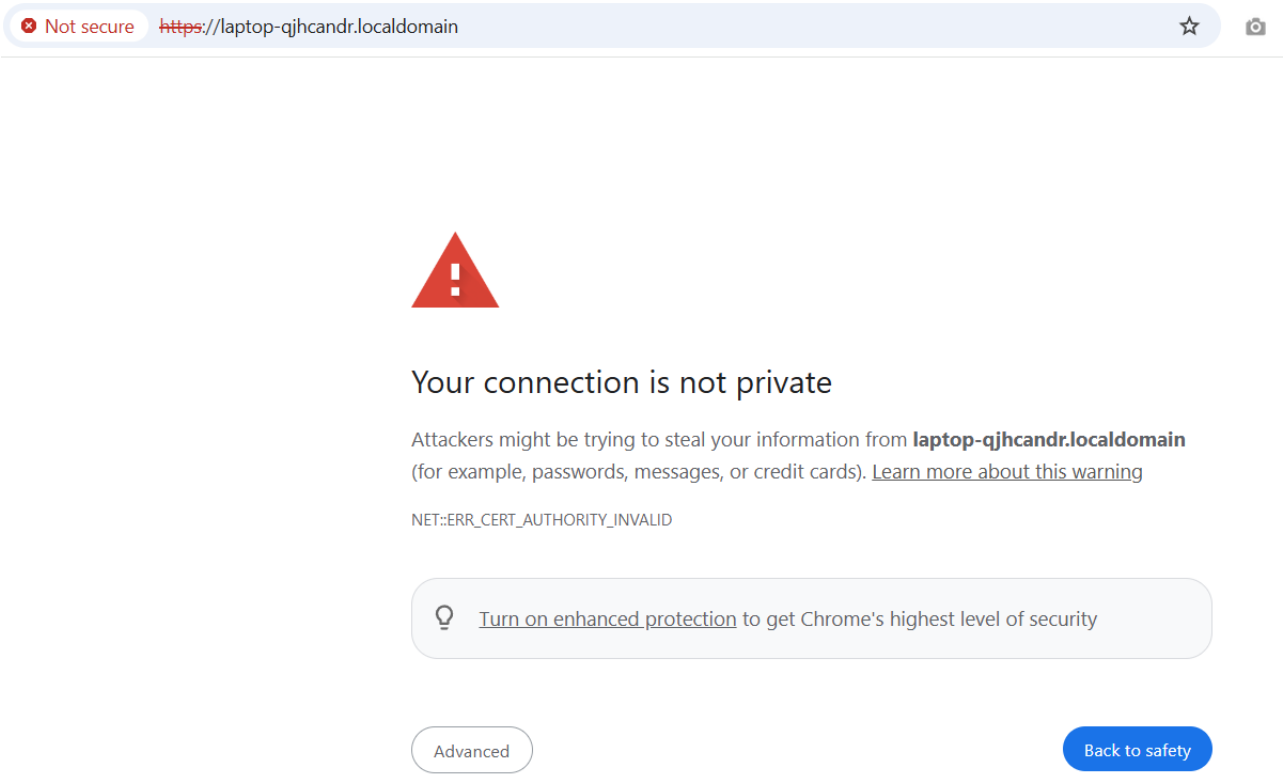
On a utilisé *John Doe* comme CA.
Cette fois, l'idée est d'obliger l'utilisateur à présenter un certificat client valide pour accéder au site HTTPS.

Maintenant on génère un fichier pkcs#12 à partir du certificat client et de sa clé privée avec la commande `openssl pkcs12 -export -in private/client.crt -inkey private/client.key -out client.p12`. Ce fichier contient à la fois le certificat client et la clé privée, et est protégé par un mot de passe et peut être importé dans un navigateur.

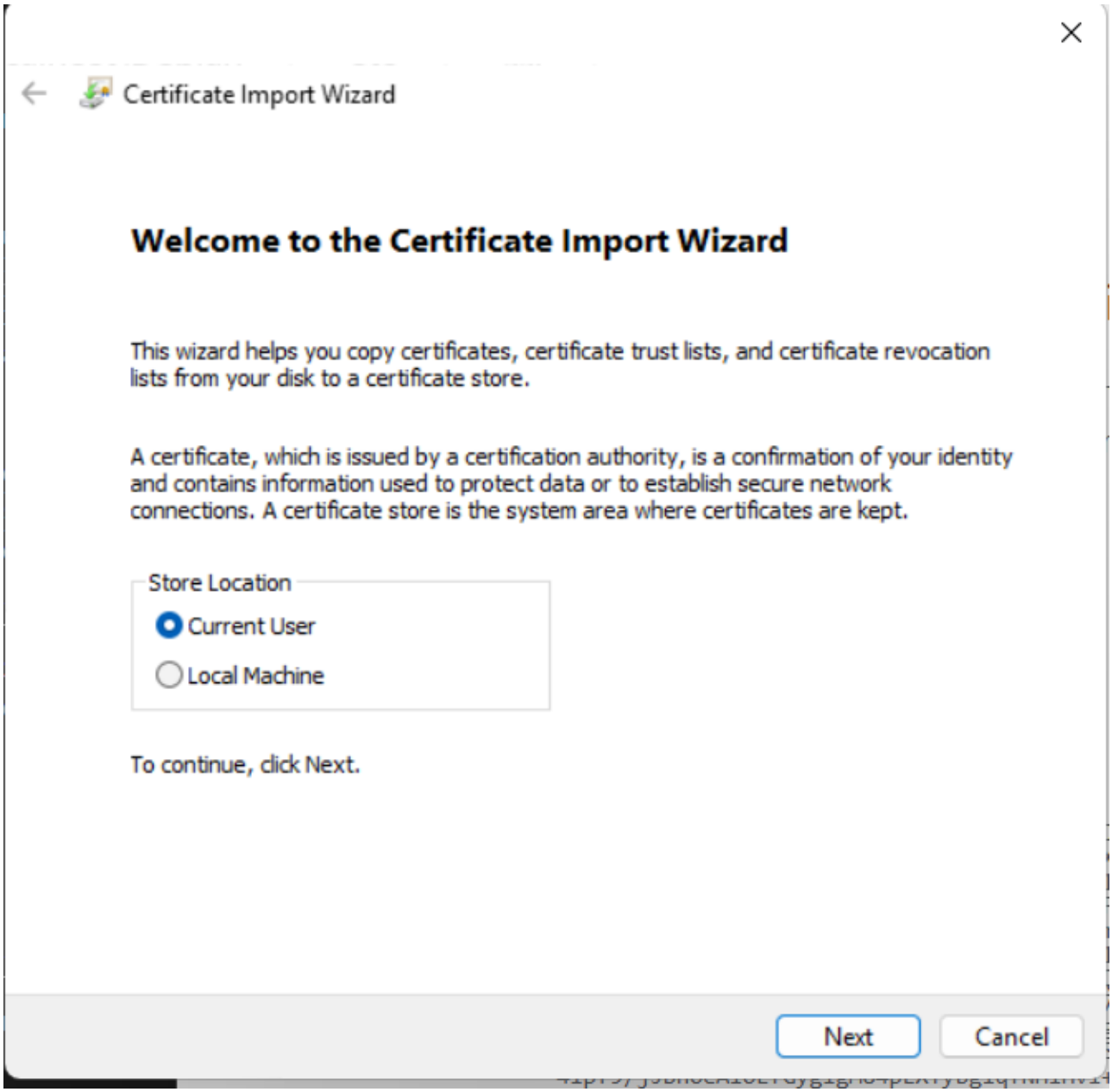
```
bluefir  ssl  sudo openssl pkcs12 -export -in private/client.crt -inkey private/client.key -out client.p12
Enter Export Password:
Verifying - Enter Export Password:
```

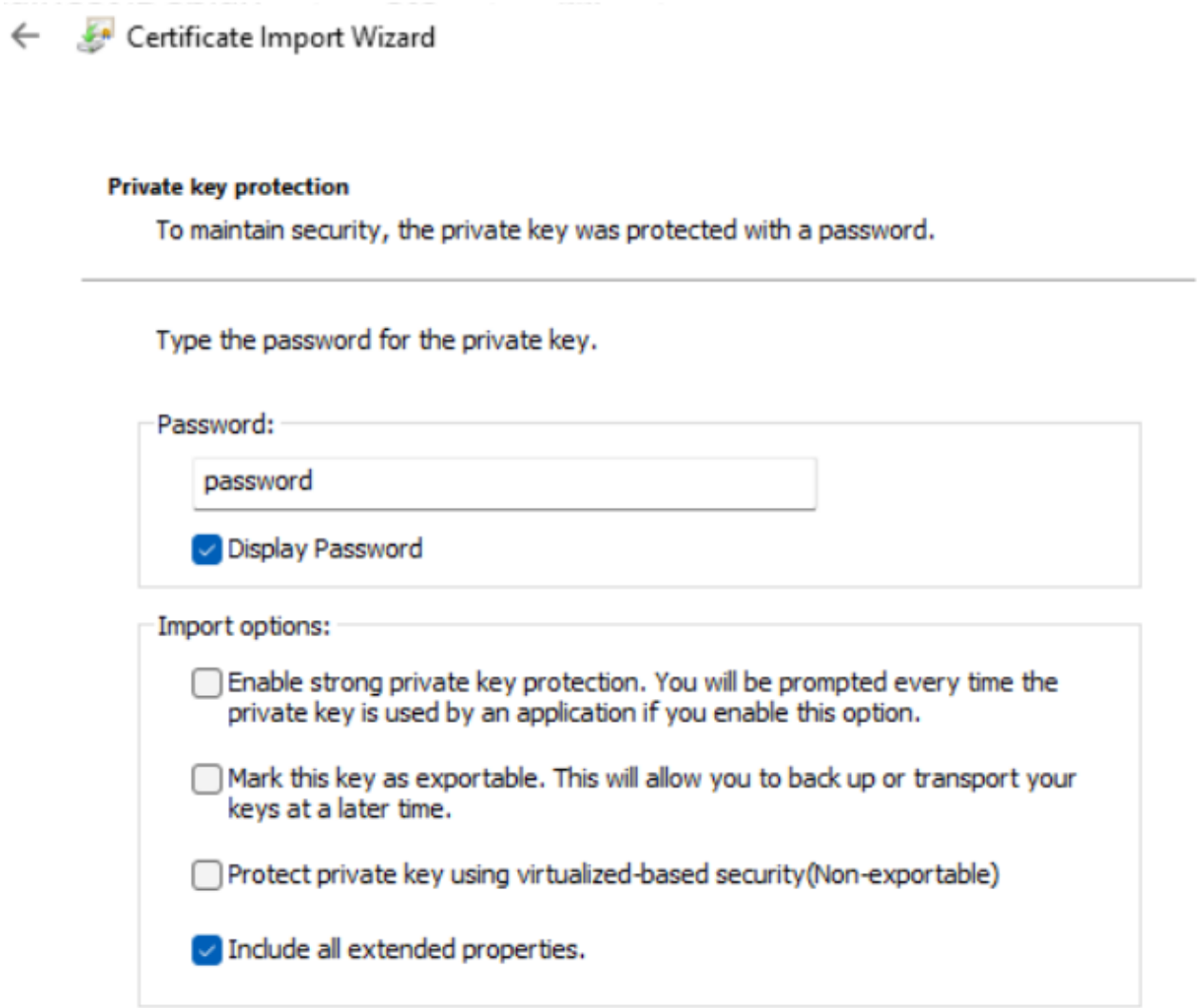
13. Après avoir ajouté dans la configuration Apache les lignes `SSLVerifyClient require`, `SSLVerifyDepth 1` et `SSLCACertificateFile /etc/ssl/private/ca.crt`, l'accès au site HTTPS échoue tant que le navigateur ne présente pas de certificat client valide. L'erreur observée est donc qu'aucun certificat client n'a été fourni, ou qu'aucun certificat acceptable n'est disponible. Selon le navigateur, cela peut apparaître comme une demande de certificat, un refus d'accès, ou un message du type *No required SSL certificate was sent*. On voit donc bien qu'ici le serveur ne se contente plus

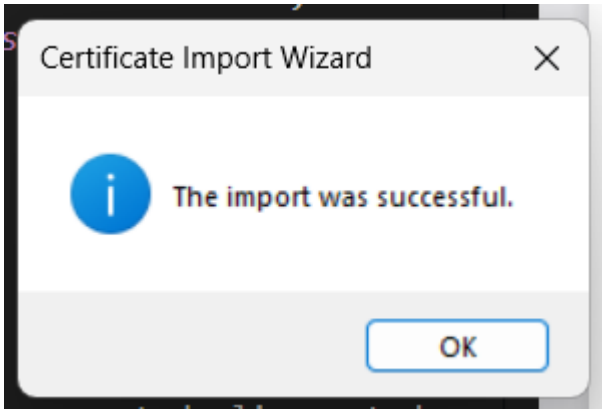
d'authentifier son identité : il vérifie aussi celle du client.



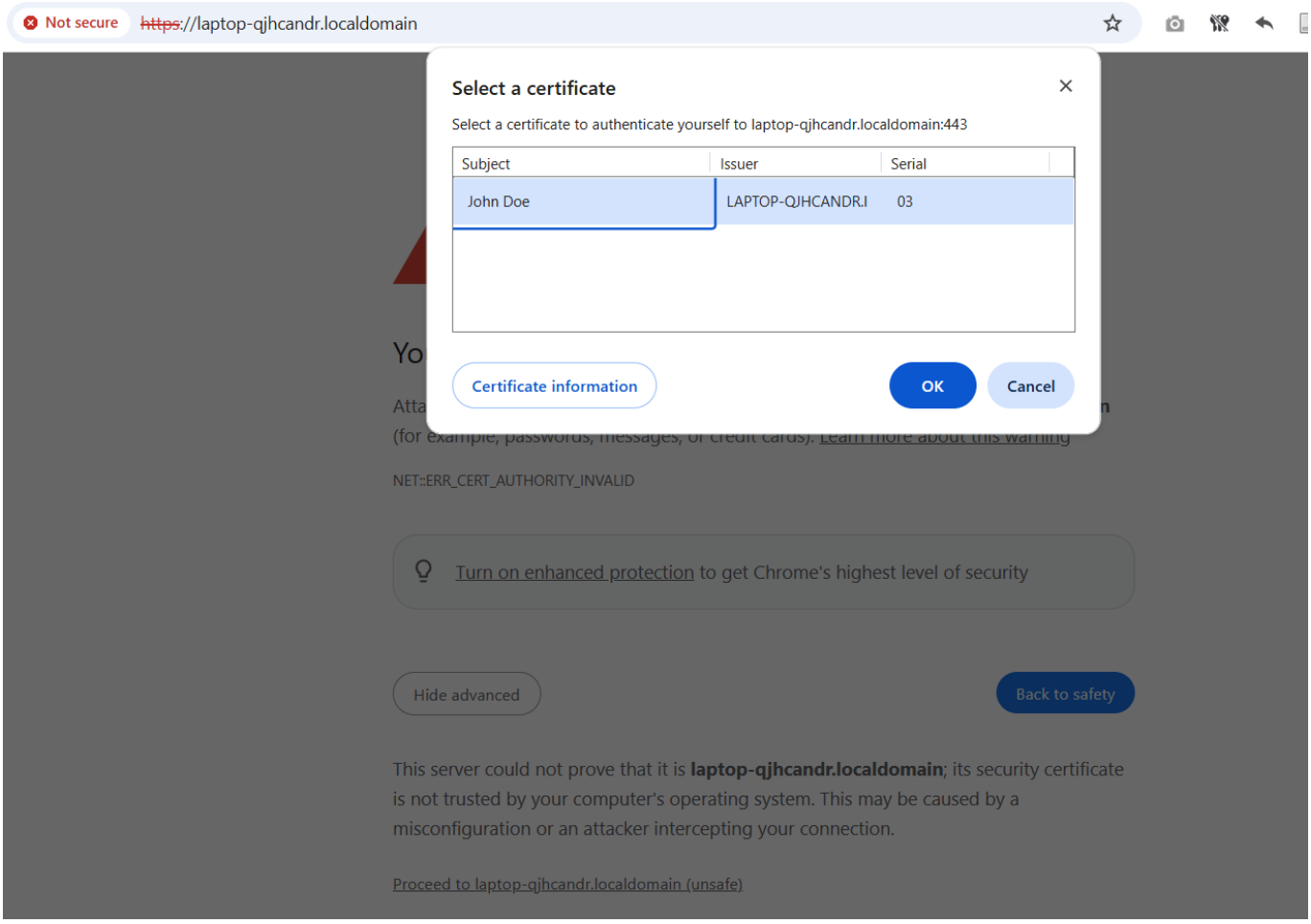
14. Lors de l'importation du fichier `client.p12` (après avoir changé ses permissions), le navigateur demande un mot de passe. Il s'agit du mot de passe défini au moment de l'export du certificat au format PKCS#12. Ce mot de passe est nécessaire car le fichier PKCS#12 contient la clé privée du client, et celle-ci doit être protégée :

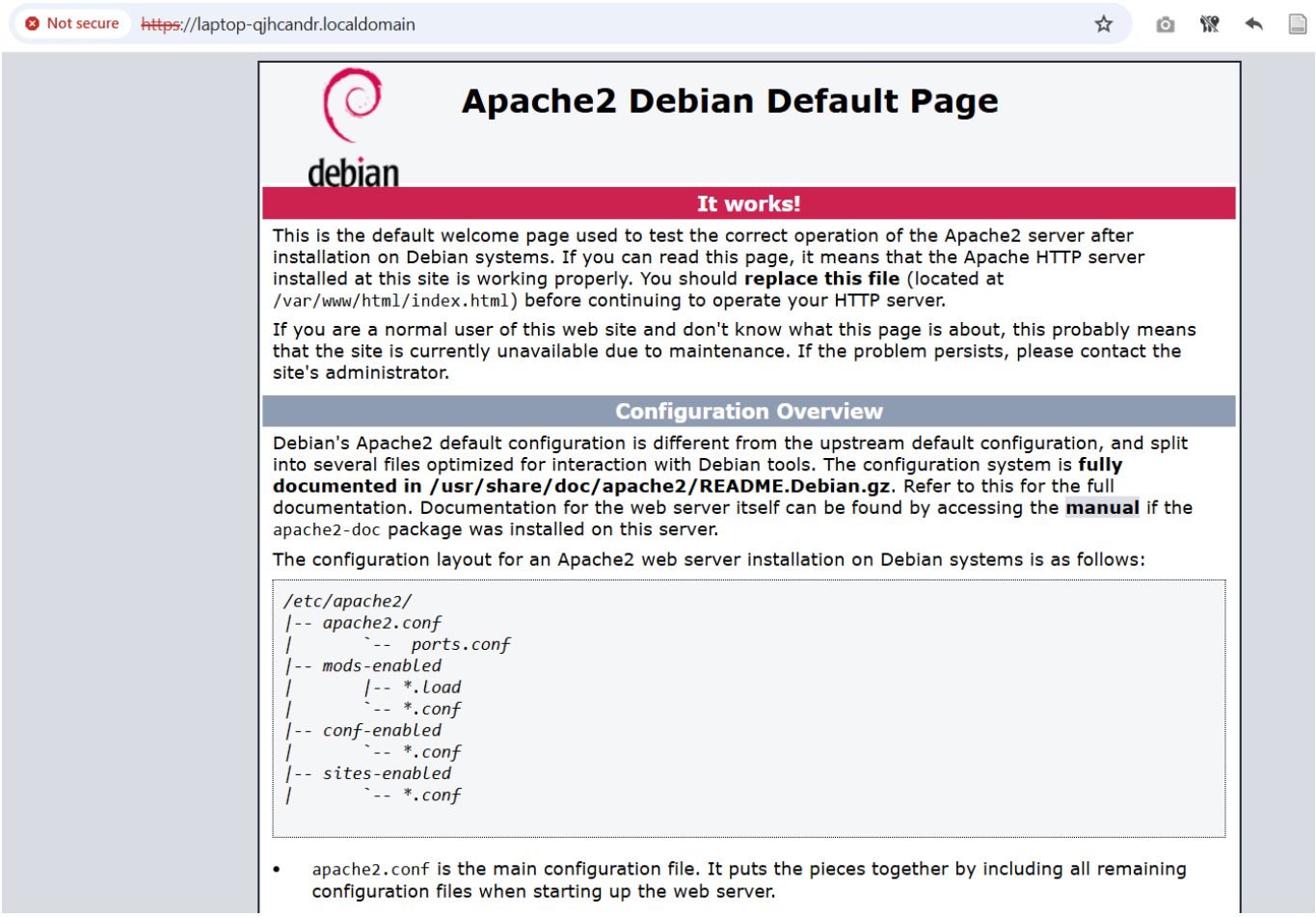

Certificate Import Wizard
Welcome to the Certificate Import Wizard
This wizard helps you copy certificates, certificate trust lists, and certificate revocation lists from your disk to a certificate store.
A certificate, which is issued by a certification authority, is a confirmation of your identity and contains information used to protect data or to establish secure network connections. A certificate store is the system area where certificates are kept.
Store Location
Current User
Local Machine
To continue, click Next.
Next
Cancel





Après l'importation, le navigateur indique normalement que le certificat a bien été ajouté aux certificats personnels, puis il peut proposer de sélectionner ce certificat lors de l'accès au site. Le résultat final est qu'une fois le bon certificat client importé et présenté au serveur, l'accès au site HTTPS est autorisé. On obtient donc cette fois une authentification mutuelle puisque le serveur prouve son identité au client, et le client prouve aussi son identité au serveur :


Not secure
https://laptop-qjhcandr.localdomain
Select a certificate
Select a certificate to authenticate yourself to laptop-qjhcandr.localdomain:443
Subject
Issuer
Serial
John Doe
LAPTOP-QJHCANDR.I
03
Certificate information
OK
Cancel
NET::ERR_CERT_AUTHORITY_INVALID
Turn on enhanced protection to get Chrome's highest level of security
Hide advanced
Back to safety
This server could not prove that it is laptop-qjhcandr.localdomain; its security certificate is not trusted by your computer's operating system. This may be caused by a misconfiguration or an attacker intercepting your connection.
Proceed to laptop-qjhcandr.localdomain (unsafe)


Not secure
https://laptop-qjhcandr.localdomain
debian
Apache2 Debian Default Page
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Debian systems is as follows:
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.

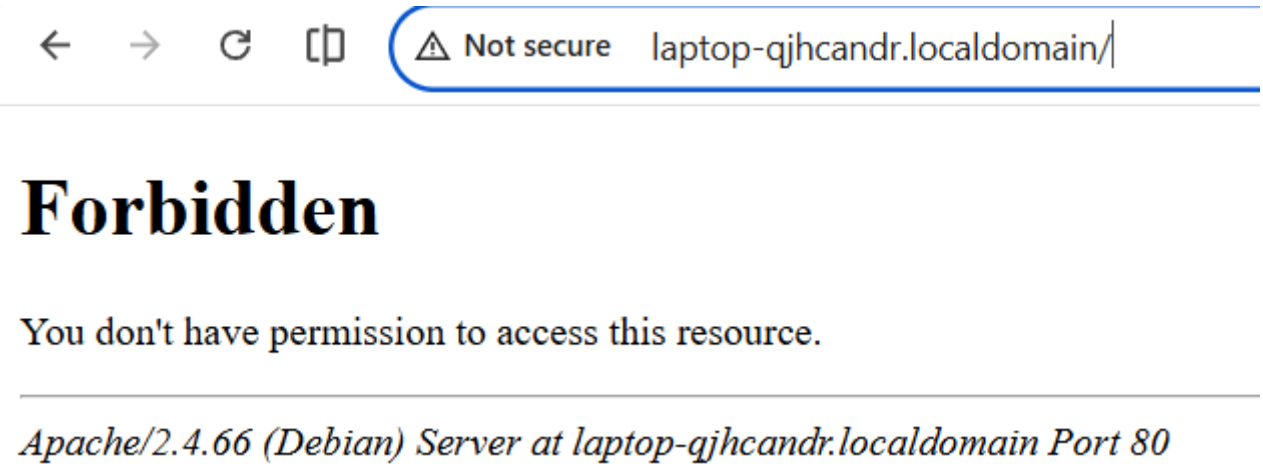
4 - RewriteEngine apache

Pour terminer, on configure le module de réécriture d'Apache afin de contrôler le comportement des connexions HTTP non sécurisées avec `a2enmod rewrite` :

```
bluefir  ssl  sudo a2enmod rewrite
Enabling module rewrite.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

15. Avec les directives suivantes dans le fichier `/etc/apache2/sites-available/000-default.conf`:

```
RewriteEngine on
RewriteCond %{HTTPS} off
RewriteRule ^/(.*) - [F]
```

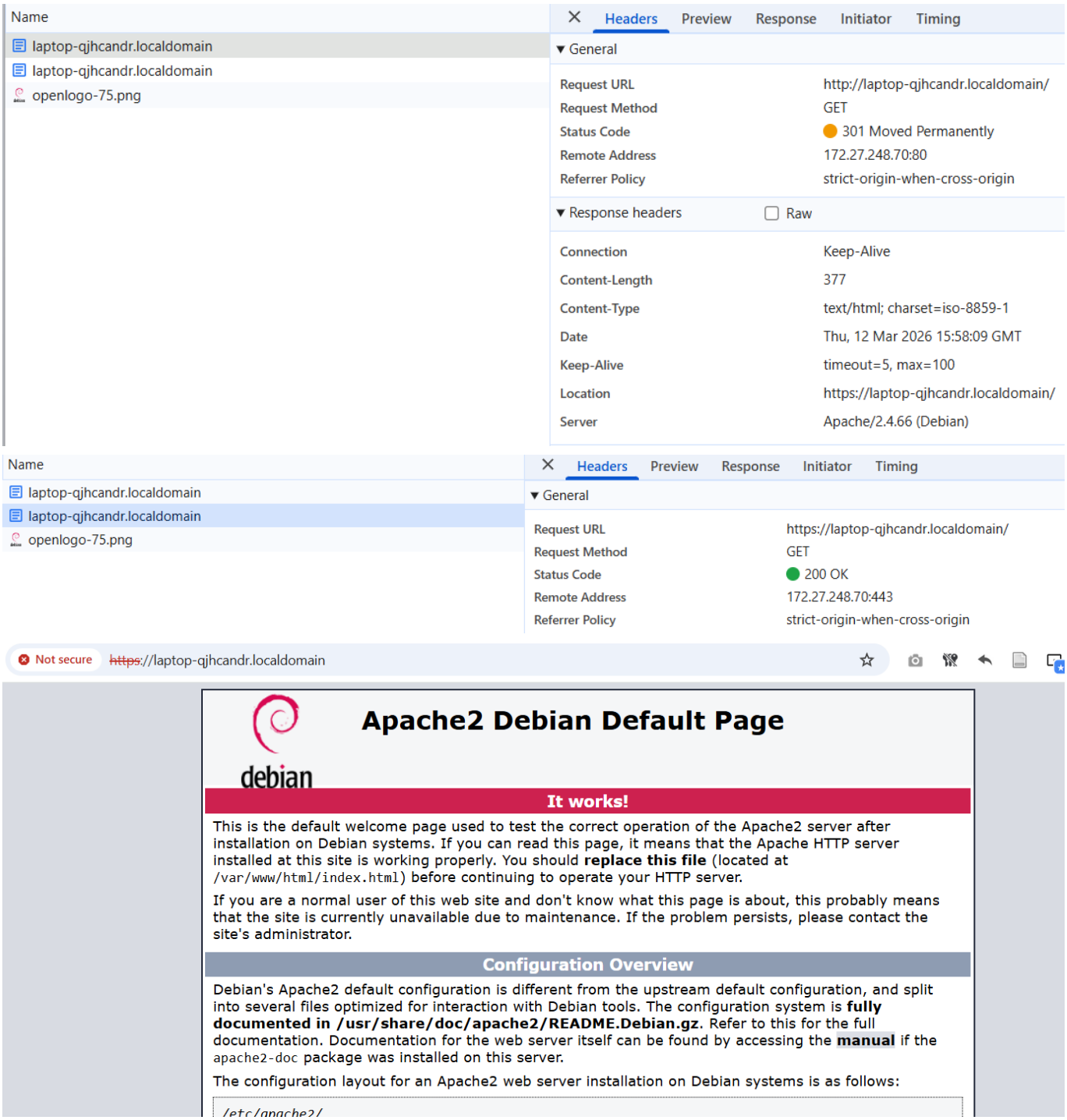


on observe que les connexions HTTP sont simplement refusées. Le résultat est donc une erreur de type **403 Forbidden** au lieu d'une redirection automatique vers HTTPS.

16. Pour forcer une connexion HTTPS au lieu de bloquer l'accès, on peut utiliser la règle de réécriture suivante :

```
RewriteEngine on
RewriteCond %{HTTPS} off
RewriteRule ^/(.*)$ https://%{HTTP_HOST}/$1 [R=301,L]
```

Cette règle redirige automatiquement toute requête HTTP vers la même ressource en HTTPS, ce qui est le comportement attendu dans un déploiement classique. C'est donc une solution plus propre qu'un simple refus d'accès, puisqu'elle guide directement l'utilisateur vers la version sécurisée du site :



Conclusion

Je conclus que ce TP permet de bien comprendre comment fonctionne une chaîne de confiance TLS, depuis la création d'une autorité de certification locale jusqu'à la signature de certificats serveur et client. Il montre aussi que la sécurité ne dépend pas seulement du chiffrement, mais également de la confiance accordée au certificat, de la correspondance entre le nom du serveur et le certificat, et de la bonne configuration d'Apache. Enfin, l'authentification par certificat client montre qu'il est possible d'aller plus loin qu'un simple HTTPS classique en imposant une authentification mutuelle. J'en retiens surtout que la partie la plus "visible" pour l'utilisateur n'est pas forcément le chiffrement lui-même, mais plutôt toute la gestion de confiance autour des certificats.